

USER_GUIDE

Containers Module - Comprehensive User Guide

Version: 2.0.0

Module: digital.vasic.containers

Go Version: 1.24+

Table of Contents

1. [Installation](#)
 2. [Quick Start](#)
 3. [Container Runtime](#)
 4. [Health Checking](#)
 5. [Compose Orchestration](#)
 6. [Remote Deployment](#)
 7. [Service Endpoints](#)
 8. [Boot Manager](#)
 9. [Lifecycle Management](#)
 10. [Event System](#)
 11. [Resource Monitoring](#)
 12. [Service Discovery](#)
 13. [Troubleshooting](#)
 14. [Best Practices](#)
-

Installation

```
go get digital.vasic.containers@latest
```

Requirements

- Go 1.24 or later
- Container runtime: Docker, Podman, or Kubernetes

- For remote deployment: SSH access with key-based authentication
-

Quick Start

Local Container Management

```
package main

import (
    "context"
    "digital.vasic.containers/pkg/runtime"
    "digital.vasic.containers/pkg/compose"
    "digital.vasic.containers/pkg/boot"
)

func main() {
    ctx := context.Background()

    // Auto-detect runtime (Docker → Podman → Kubernetes)
    rt, _ := runtime.AutoDetect(ctx)

    // Create boot manager
    manager := boot.NewBootManager(
        boot.WithRuntime(rt),
    )

    // Add services
    manager.AddService("postgres", boot.ServiceConfig{
        ComposeFile: "docker-compose.yml",
        HealthCheck: boot.TCPCheck("localhost", 5432),
        Required:    true,
    })

    // Start all services
    manager.Start(ctx)
}
```

Remote Deployment

```
package main

import (
    "context"
    "digital.vasic.containers/pkg/remote"
    "digital.vasic.containers/pkg/compose"
)
```

```

func main() {
    ctx := context.Background()

    host := remote.RemoteHost{
        Name:      "production",
        Address:    "prod.example.com",
        User:      "deploy",
        Runtime:    "podman",
    }

    executor, _ := remote.NewSSHExecutor(nil)
    orch := remote.NewRemoteComposeOrchestrator(host, executor,
        nil)

    project := compose.ComposeProject{
        File: "/path/to/docker-compose.yml",
        Name: "myapp",
    }
    orch.Up(ctx, project)
}

```

Container Runtime

Auto-Detection

```

import "digital.vasic.containers/pkg/runtime"

ctx := context.Background()

// Auto-detect available runtime
rt, err := runtime.AutoDetect(ctx)
if err != nil {
    // No runtime available
}

// Runtime name: "docker", "podman", or "kubernetes"
fmt.Println("Detected:", rt.Name())

```

Specific Runtime

```

// Force Docker
docker, err := runtime.NewDockerRuntime()

// Force Podman
podman, err := runtime.NewPodmanRuntime()

// Force Kubernetes
k8s, err := runtime.NewKubernetesRuntime()

```

Runtime Operations

```
// Version
version, _ := rt.Version(ctx)

// List containers
containers, _ := rt.List(ctx, runtime.ListFilter{All: true})

// Get container status
status, _ := rt.Status(ctx, "my-container")

// Execute command in container
result, _ := rt.Exec(ctx, "my-container", []string{"echo",
    "hello"})

// Start/Stop/Remove
rt.Start(ctx, "my-container")
rt.Stop(ctx, "my-container", 10*time.Second)
rt.Remove(ctx, "my-container")
```

Health Checking

TCP Health Check

```
import "digital.vasic.containers/pkg/health"

checker := health.NewDefaultChecker()

result := checker.Check(ctx, health.HealthTarget{
    Name:    "postgres",
    Host:    "localhost",
    Port:    "5432",
    Type:    health.HealthTCP,
    Timeout: 5 * time.Second,
})

if result.Healthy {
    fmt.Println("PostgreSQL is healthy")
}
```

HTTP Health Check

```
result = checker.Check(ctx, health.HealthTarget{
    Name:    "api",
    Host:    "localhost",
    Port:    "8080",
    Type:    health.HealthHTTP,
    Path:    "/health",
})
```

```
    Timeout: 5 * time.Second,  
})
```

Retry Policy

```
policy := health.RetryPolicy{  
    MaxRetries: 5,  
    Delay: 2 * time.Second,  
    BackoffFactor: 1.5,  
}  
result = health.CheckWithRetry(ctx, checker, target, policy)
```

Compose Orchestration

Basic Usage

```
import "digital.vasic.containers/pkg/compose"  
  
orch := compose.NewDefaultOrchestrator()  
  
project := compose.ComposeProject{  
    Name: "myapp",  
    File: "docker-compose.yml",  
    Profile: "default",  
    Services: []string{"postgres", "redis"},  
}  
  
// Start services  
err := orch.Up(ctx, project, compose.WithDetach(true))  
  
// Check status  
statuses, _ := orch.Status(ctx, project)  
for _, s := range statuses {  
    fmt.Printf("%s: %s (health: %s)\n", s.Name, s.State, s.Health)  
}  
  
// Stop services  
err = orch.Down(ctx, project)
```

With Profiles

```
project := compose.ComposeProject{  
    File: "docker-compose.yml",  
    Profile: "production",  
}  
orch.Up(ctx, project)
```

Remote Deployment

Host Configuration

```
import "digital.vasic.containers/pkg/remote"

host := remote.RemoteHost{
    Name:      "prod-server",
    Address:   "192.168.1.100",
    Port:      22,
    User:      "deploy",
    Runtime:   "podman",
    Labels:    map[string]string{
        "gpu":      "true",
        "memory":   "high",
    },
}
```

SSH Executor

```
executor, err := remote.NewSSHExecutor(
    logger,
    remote.WithControlMaster(true),
    remote.WithControlPersist(600*time.Second),
    remote.WithConnectTimeout(10*time.Second),
    remote.WithCommandTimeout(60*time.Second),
)
defer executor.Close()

// Execute command
result, err := executor.Execute(ctx, host, "podman ps")

// Copy file
err = executor.CopyFile(ctx, host, "/local/file", "/remote/path")
```

Remote Compose Orchestrator

```
orch := remote.NewRemoteComposeOrchestrator(host, executor,
    logger)

// Auto-detects compose command: podman-compose > docker compose >
//                               podman compose > docker-compose
project := compose.ComposeProject{
    File: "/local/docker-compose.yml",
}

// Deploy
err := orch.Up(ctx, project)

// Check status
```

```
statuses, _ := orch.Status(ctx, project)
```

```
// Teardown
```

```
orch.Down(ctx, project)
```

Compose Command Detection

The module automatically detects the best compose command:

Prior-ity	Command	Notes
1	podman-compose	Native Podman compose (preferred)
2	docker-compose	Docker v2 plugin
3	podman-compose	Podman wrapper (may use docker-compose v1)
4	docker-compose	Legacy v1 standalone

Scheduler Strategies

```
import "digital.vasic.containers/pkg/scheduler"
```

```
// Resource-aware (default)
```

```
sched := scheduler.NewScheduler(hostManager, logger,  
    scheduler.WithStrategy(scheduler.StrategyResourceAware),  
)
```

```
// Strategies available:
```

```
// - StrategyResourceAware: Score-based distribution
```

```
// - StrategyRoundRobin: Even distribution
```

```
// - StrategyAffinity: Label-based placement
```

```
// - StrategySpread: Maximize distribution
```

```
// - StrategyBinPack: Fill hosts first
```

Service Endpoints

```
import "digital.vasic.containers/pkg/endpoint"
```

```
ep := endpoint.NewEndpoint().  
    WithHost("localhost").  
    WithPort("5432").  
    WithEnabled(true).  
    WithRequired(true).  
    WithHealthType("tcp").
```

```
WithTimeout(10 * time.Second).
WithRetryCount(5).
WithComposeFile("docker-compose.yml").
WithServiceName("postgres").
Build()
```

```
// Resolved URL
```

```
url := ep.ResolvedURL() // "localhost:5432"
```

Boot Manager

```
import "digital.vasic.containers/pkg/boot"
```

```
manager := boot.NewBootManager(
    boot.WithRuntime(rt),
    boot.WithLogger(logger),
    boot.WithHealthCheckRetries(5),
    boot.WithParallelStartup(true),
)
```

```
// Add services
```

```
manager.AddService("postgres", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    ServiceName: "postgres",
    HealthCheck: boot.TCPCheck("localhost", 5432),
    HealthTimeout: 10 * time.Second,
    Required: true,
})
```

```
// Start all
```

```
summary, err := manager.Start(ctx)
```

```
// Summary contains: started, failed, skipped, remote counts
```

```
// Shutdown
```

```
manager.Shutdown(ctx)
```

Lifecycle Management

```
import "digital.vasic.containers/pkg/lifecycle"
```

```
lm := lifecycle.NewDefaultManager(orchestrator, checker)
```

```
// Register service with lazy boot
```

```
lm.Register(lifecycle.ServiceSpec{
    Name: "postgres",
    LazyBoot: true,
})
```



```

        IdleTimeout: 30 * time.Minute,
        MaxConcurrent: 10,
        Priority: 1,
        HealthTarget: pgHealthTarget,
    })

    // Acquire (starts if lazy and not running)
    release, err := lm.Acquire(ctx, "postgres")
    defer release()

    // Use the service...

    // Release decrements usage count (stops if idle)

```

Event System

```

import "digital.vasic.containers/pkg/event"

bus := event.NewEventBus(100)

// Subscribe to events
id := bus.Subscribe(
    event.EventFilter{Types:
        []event.EventType{event.EventContainerStarted}},
    func(ctx context.Context, e event.Event) {
        fmt.Printf("Container started: %s\n", e.Name)
    },
)

// Publish event
bus.Publish(ctx, event.Event{
    Type: event.EventContainerStarted,
    Name: "postgres",
})

// Unsubscribe
bus.Unsubscribe(id)

```

Event Types

Event	Description
EventContainerStarted	Container started
EventContainerStopped	Container stopped
EventHealthCheckFailed	Health check failed
EventHostOnline	Remote host online
EventHostOffline	Remote host offline

Event	Description
EventDeployed	Service deployed
EventTunnelCreated	SSH tunnel created

Resource Monitoring

```
import "digital.vasic.containers/pkg/monitor"

mon := monitor.NewDefaultMonitor(rt)

// Set threshold alerts
mon.SetThreshold(monitor.ThresholdRule{
    Metric:    "cpu_percent",
    Threshold: 90.0,
    Operator:  ">",
    Action: func(snap *monitor.ResourceSnapshot) {
        fmt.Println("CPU usage too high!")
    },
})

// Start monitoring (every 10 seconds)
mon.Start(ctx, 10*time.Second)
defer mon.Stop()

// Get snapshot
snap, _ := mon.Snapshot()
fmt.Printf("CPU: %.2f%%, Memory: %.2f%%\n", snap.CPUPercent,
    snap.MemoryPercent)
```

Service Discovery

```
import "digital.vasic.containers/pkg/discovery"

// TCP port scanning
scanner := discovery.NewTCPScanner(nil)
services, _ := scanner.Scan(ctx, "192.168.1.0/24", []int{5432,
    6379, 8080})

// DNS discovery
resolver := discovery.NewDNSResolver(nil)
addrs, _ := resolver.Lookup(ctx, "postgres.service.consul")

// mDNS discovery
mdns := discovery.NewMDNSDiscovery(nil)
services, _ := mdns.Discover(ctx, "_postgres._tcp")
```

Troubleshooting

Common Issues

"no compose command found"

Solution:

```
# Install podman-compose (recommended)
pip install podman-compose
```

```
# Or Docker Compose v2
sudo apt install docker-compose-plugin
```

"Not supported URL scheme http+docker"

Cause: podman compose delegates to incompatible docker-compose v1.

Solution: Install podman-compose:

```
pip install podman-compose
```

"connection refused"

Solutions:

1. Verify SSH key: `ssh-add -l`
 2. Test manually: `ssh deploy@host`
 3. Check firewall: port 22
-

Best Practices

1. Use Podman for Remote Deployment

```
CONTAINERS_REMOTE_HOST_1_RUNTIME=podman
```

2. Enable SSH ControlMaster

```
CONTAINERS_REMOTE_CONTROL_MASTER=true
CONTAINERS_REMOTE_CONTROL_PERSIST=600
```

3. Always Configure Health Checks

```
manager.AddService("postgres", boot.ServiceConfig{
    HealthCheck: boot.TCPCheck("localhost", 5432),
```

```
    Required:    true,  
  })
```

4. Use Resource Labels

```
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=true,memory=high
```

5. Monitor Resources

```
monitor := monitor.NewResourceMonitor(runtime, nil)  
go monitor.Start(ctx)
```

API Reference

See [API_REFERENCE.md](#) for complete API documentation.

Architecture

See [ARCHITECTURE.md](#) for architecture details.

Remote Deployment

See [REMOTE_DEPLOYMENT.md](#) for remote deployment guide.

Last Updated: February 2026